SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2- DEBUGGING & OPTIMISATION TASK

Course Code:	CSA1223	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL3- DEBUGGING & OPTIMISATION TASK
Weightage:	20%	Total Marks:	100
CO2 Statement:	*Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)*		
Student Name:	S.M.SNEKA	Reg.No.:	192321095
Department:	IT	Date:	

16/.

ANNEXURE A

1. A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.

2. A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.

3. A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.

4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.

5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

Code of Conduct:

I S.M. SNEKA (192321095) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: Sneka.

MARKS ALLOCATION

	Identification of Errors / Bugs (20%)	Debugging Approach & Analysis (20%)	Correctness of Debugged Solution (25%)	Optimization Techniques Applied (20%)	Testing and Documentation (15%)	
Q1	3	3	2	3	3	14
Q2	4	3	2	3	2	14
Q3	3	4	3	4	2	16
Q4	2	3	4	3	3	15
Q5	3	3	5	3	3	17
						76

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Identification of Errors / Bugs	20%	Accurately identifies all errors and issues with complete understanding of the problem(4)	Identifies most errors with minor omissions(3)	Identifies limited errors; misses key issues(2)	Unable to identify errors or misunderstands the problem(1)
Debugging Approach & Analysis	20%	Uses effective debugging techniques with clear analysis and root cause identification(4)	Applies suitable debugging methods with minor gaps(3)	Uses basic debugging methods with limited analysis(2)	No systematic debugging approach followed(0)
Correctness of Debugged Solution	25%	Provides completely corrected solution with accurate functionality and expected output(5)	Provides working solution with minor errors(4)	Partially correct solution with limited functionality(3)	Incorrect solution or fails to resolve errors(0)
Optimization Techniques Applied	20%	Applies efficient optimization methods to improve performance, memory usage and quality(4)	Performs optimization with noticeable improvements(3)	Limited optimization with minimal improvements(2)	No optimization applied or inefficient implementation(0)
Testing and Documentation	15%	Performs complete testing with proper validation and clear documentation(3)	Provides adequate testing and documentation with minor issues(2)	Limited testing and incomplete documentation(1)	No proper testing or documentation provided(0(

1)A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.

ANSWER:

In 8-bit 2's complement representation, the range of signed integers is –128 to +127. Therefore, +120 and +50 cannot both be represented by their mathematical sum because 120 + 50 = 170, which is greater than +127.

Binary Representation:

+120 = 01111000

+50 = 00110010

Binary Addition:

01111000

+ 00110010

10101010

The result 10101010 has its most significant bit set to 1, so it represents a negative number in 2's complement. Its magnitude is obtained by taking the 2's complement:

10101010 → 01010101 → 01010110 = 86

Hence, the bit pattern represents –86, not +170.

Bug Identification:

The error is signed overflow. Adding two positive numbers produced a negative result. This occurs because the mathematical result is outside the representable 8-bit signed range.

Overflow Detection:

Overflow occurs when two operands have the same sign but the result has the opposite sign. It can also be detected using: Overflow = Carry into MSB XOR Carry out of MSB.

Solution:

The processor/program should check the overflow flag after addition. If overflow is detected, the result must be reported as outside the valid 8-bit signed range or the operation should be performed using a wider data type such as 16 bits.

Testing:

+120 + +50 → overflow detected.

+40 + +30 → 011? valid positive result (+70).

–100 + –40 → overflow detected.

+60 + –20 → +40, no overflow.

Conclusion:

The binary addition itself is correct; the problem is interpreting an out-of-range result as a valid signed value. Proper overflow detection solves the issue.

- **A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.**

Problem Analysis:

A ripple carry adder (RCA) calculates each bit sequentially. The carry generated at a lower-order bit must propagate to the next higher-order bit before that bit can produce its final sum. Therefore, the total delay increases with the number of bits.

For each bit:

Pi = Ai XOR Bi (propagate)

Gi = Ai AND Bi (generate)

Si = Pi XOR Ci

In a ripple carry design:

C1 depends on C0,

C2 depends on C1,

C3 depends on C2,

and so on.

For an n-bit adder, the worst-case carry may have to pass through all n stages. This creates a significant delay in real-time applications.

Bug/Performance Bottleneck:

The main bottleneck is serial carry propagation. The arithmetic result is correct, but the hardware implementation is slow because each stage waits for the previous carry.

Optimized Solution – Carry Look-Ahead Adder (CLA):

A CLA calculates carries in parallel using generate and propagate signals.

For example:

$$C1 = G0 + P0C0$$

$$C2 = G1 + P1G0 + P1P0C0$$

$$C3 = G2 + P2G1 + P2P1G0 + P2P1P0C0$$

Thus, higher-order carries can be determined without waiting for every intermediate stage.

Performance Improvement:

RCA: carry propagates stage by stage → larger propagation delay.

CLA: carries are generated using parallel logic → much smaller delay.

Testing:

Compare an 8-bit RCA and 8-bit CLA for the same input combinations. Both must produce identical sums and carry outputs, while the CLA should have lower propagation delay.

Conclusion:

Replacing the ripple carry structure with a carry look-ahead structure reduces carry propagation delay and is suitable for high-speed arithmetic and real-time systems.

3)A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug t step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify th possible source of error and suggest corrections to ensure accurate signed multiplication.

Problem Analysis:

Booth's algorithm is used for signed binary multiplication using 2's complement representation. It examines the pair (Q0, Q-1) during each iteration.

Decision Table:

Q0 Q-1 = 01 → A = A + M

Q0 Q-1 = 10 → A = A – M

Q0 Q-1 = 00 → No arithmetic operation

Q0 Q-1 = 11 → No arithmetic operation

After the operation, the combined register A-Q-Q-1 is subjected to an arithmetic right shift.

Possible Bugs:

1. Checking the wrong bit pair instead of Q0 and Q-1.
2. Using a logical right shift instead of an arithmetic right shift.
3. Failing to sign-extend the accumulator and multiplicand.
4. Incorrectly performing A – M without 2's complement subtraction.
5. Performing the wrong number of iterations.
6. Incorrectly initializing Q-1 instead of setting it to 0.

Debugging Procedure:

Initialize:

A = 0

Q = multiplier

M = multiplicand

Q-1 = 0

Count = number of bits

For every iteration:

1. Inspect Q0 and Q-1.
2. Apply the Booth operation from the decision table.

3. Perform an arithmetic right shift on A-Q-Q-1.

4. Decrement the count.

5. Repeat until the required number of iterations is completed.

Sign Extension:

During an arithmetic right shift, the sign bit of A must be copied into the new MSB. This preserves the 2's complement sign.

Example Test:

For signed multiplication such as (–5) × (+3), the expected decimal result is –15. The final 2's complement product must represent –15.

Testing:

Test all sign combinations:

(+5) × (+3) = +15

(–5) × (+3) = –15

(+5) × (–3) = –15

(–5) × (–3) = +15

Conclusion:

Most incorrect Booth results for negative operands are caused by incorrect sign extension, logical shifting, wrong Q0/Q-1 checking, or incorrect subtraction. Correct initialization and arithmetic shifting ensure accurate signed multiplication.

4)An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration step leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occu Propose an optimized approach using the non-restoring division method, explaining how it improves performance.

Problem Analysis:

In the restoring division algorithm, the divisor is subtracted from the partial remainder. If the result becomes negative, the divisor must be added back to restore the previous remainder. This restoration step may occur repeatedly and increases execution time.

Restoring Division Workflow:

1. Shift the partial remainder and quotient.

2. Subtract the divisor.

3. If the result is negative, restore the previous remainder by adding the divisor.

4. Set the quotient bit accordingly.

5. Repeat for all bits.

Performance Bottleneck:

The unnecessary restore operation requires an additional addition whenever the subtraction produces a negative partial remainder.

Optimized Solution – Non-Restoring Division:

Non-restoring division avoids immediately restoring the remainder. Instead:

- If the current remainder is positive, subtract the divisor in the next operation.
- If the current remainder is negative, add the divisor in the next operation.
- The quotient bit is determined from the sign of the partial remainder.

Basic Algorithm:

1. Shift the partial remainder and quotient.
2. If the previous remainder was positive, subtract the divisor.
3. If the previous remainder was negative, add the divisor.
4. Set the quotient bit based on the resulting sign.
5. Repeat for all bits.
6. If the final remainder is negative, perform a final correction by adding the divisor.

Optimization:

The major improvement is the elimination of repeated restoration after every unsuccessful subtraction. This reduces arithmetic operations and improves execution time.

Testing:

For a simple unsigned example such as 13 ÷ 3:

Expected quotient = 4

Expected remainder = 1

Both restoring and non-restoring methods should produce the same final result.

Conclusion:

Non-restoring division improves performance by avoiding repeated restore operations while maintaining correct quotient and remainder values.

5)A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results wh adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to impr accuracy.

Problem Analysis:

IEEE 754 single precision uses:

1 sign bit + 8 exponent bits + 23 fraction bits.

A very large number and a very small number may have significantly different exponents. Before addition, the number with the smaller exponent must be shifted right so that both numbers have the same exponent.

Debugging Steps:

1. Compare the exponents.
2. Shift the significand of the number with the smaller exponent to align the exponents.

3. Add or subtract the significands depending on their signs.
4. Normalize the resulting significand.
5. Adjust the exponent after normalization.
6. Round the result according to the IEEE 754 rounding mode.
7. Check for overflow, underflow, zero, infinity, and NaN where applicable.

Source of Inaccuracy:

When a very small number is added to a very large number, exponent alignment may shift the small number so far to the right that its significant bits are lost. This is called loss of significance or absorption. Single precision also has limited precision, so repeated floating-point operations can accumulate rounding errors.

Example:

A large value such as 1.0×10^8 combined with a much smaller value such as 1.0×10^0 may not preserve the small contribution accurately in single precision.

Optimization Techniques:

1. Use double precision when higher numerical accuracy is required.
2. Reorder calculations to reduce cancellation and loss of significance.
3. Use numerically stable algorithms.
4. Use guard, round, and sticky bits during intermediate calculations.
5. Avoid unnecessary conversion between integer, single precision, and double precision.
6. Use compensated summation techniques for long sequences of floating-point additions.

Testing:

Compare single-precision and double-precision results for values with large exponent differences. Check whether the small value affects the final result and measure the difference from a higher-precision reference.

Conclusion:

The inaccurate result is mainly caused by limited precision, exponent alignment, and rounding. Double precision and numerically stable algorithms can significantly improve accuracy.